CS6230 — CAD for VLSI

QR Code Decoder Project

RTL Microarchitecture and Verification Report

Full-specification QR Code Decoder *mkQRDecoder*, Versions 1–40

Student name	Kishore (Kishore Kotapati)
Roll number	EE26E016
Team partner	Rishi (EE25S082)
Department / programme	Electrical and Electronics Engineering, B.Tech
Institution	National Institute of Technology Manipur (B.Tech EEE)
Course	CS6230 — CAD for VLSI (CS6230)
Faculty / supervisor	[Faculty / course instructor — to be filled in]
Design	mkQRDecoder (Bluespec SystemVerilog), SAED 14nm RVT
Submission date	9 November 2026
Version	Final submission v1.0

Companion to **pd_report.pdf** (physical design and PPA).

Abstract

This report documents the RTL micro-architecture, implementation and verification of **mkQRDecoder**, a hardware QR-code decoder written in Bluespec SystemVerilog for CS6230. The decoder supports all 40 versions, all four error-correction levels, all eight masks, numeric, alphanumeric and byte modes, BCH(15,5) correction of the format information (up to 3 wrong bits) and Reed–Solomon correction over GF(2^8) of every block, and detects uncorrectable symbols without miscorrection.

Headline results. Verification: **76 Python reference tests** (8 files) and **9 Bluespec benches** pass under one command (`make test`); two independent **120-case full-decoder hardware sweeps** (240 symbols in total: clean, damaged to the limit, over the limit) give **120/120 with 0 mismatches each**. Latency: a version-40 symbol decodes in **38,074 cycles** (45.69 µs at the final 1.20 ns clock); the handout version-3 symbol in 563 cycles. Physical design (detailed in pd_report.pdf): 5.00 ns → **1.20 ns (833.3 MHz)**, post-place 0 violating paths, 92.29% utilisation.

Keywords: QR code, Reed–Solomon, Berlekamp–Massey, GF(2^8), Bluespec, Bluesim, verification, hardware decoder.

Table of Contents

Chapter 1 Introduction . . . 6

1.1 Background and problem statement . . . 6

1.2 Objectives and scope . . . 6

1.3 Report organisation . . . 6

Chapter 2 Assignment Specification and Requirements . . . 6

2.1 What the specification demands . . . 6

2.2 Test-bench guarantees (A1–A5) . . . 7

2.3 Requirement traceability matrix . . . 7

Chapter 3 Theoretical Background . . . 8

3.1 QR symbol structure . . . 8

3.2 Format information and BCH(15,5) . . . 8

3.3 Masks and reading order . . . 8

3.4 Blocks, interleaving and Reed–Solomon . . . 8

3.5 Payload . . . 8

Chapter 4 System Architecture and Design . . . 8

4.1 Overall architecture . . . 8

4.2 Data and control flow . . . 9

4.3 Design decisions . . . 9

4.4 Micro-architectural evolution across iterations . . . 10

Chapter 5 Development Environment and Tools . . . 10

Chapter 6 Implementation . . . 11

6.1 Source-file inventory . . . 11

6.2 Module interfaces . . . 11

6.3 Key implementation points . . . 11

Chapter 7 Command-by-Command Workflow . . . 12

7.1 Build commands . . . 12

7.2 Bluesim benches . . . 12

7.3 One-command regression . . . 12

7.4 Hardware sweeps . . . 13

7.5 Packaging and integrity . . . 13

Chapter 8 Verification and Testing . . . 14

8.1 Methodology . . . 14

8.2 Tier 1: Python reference and protocol tests (8 files, 76 tests) . . . 14

8.3 Tier 2: Bluespec benches (9 in make test) . . . 14

8.4 Tier 3: whole-decoder hardware sweeps . . . 15

8.5 Coverage of the required test categories . . . 15

8.6 Complete verification inventory . . . 15

8.7 Known limits of the verification . . . 17

Chapter 9 Results and Analysis . . . 17

9.1 All timing results (ns) — summary table . . . 18

9.2 Why the faster periods (down to 0.55 ns) were not chosen . . . 18

9.3 Best-design data sheet . . . 18

9.4 Discussion of the results . . . 19

Chapter 10 Challenges, Debugging and Solutions . . . 19

Chapter 11 Discussion . . . 20

Chapter 12 Conclusion and Future Work . . . 20

Appendix A Complete Command Reference . . . 20

Appendix B Screenshot Evidence Index . . . 21

Appendix C File and Directory Inventory . . . 21

Appendix D Glossary and Abbreviations . . . 21

Appendix E References . . . 22

List of Figures

Figure 4.1 — Block diagram of mkQRDecoder and its surroundings . . . 9

Figure 4.2 — Project directory structure [SS-R01] . . . 10

Figure 5.1 — Tool versions on the server [SS-R02] . . . 10

Figure 6.1 — Sources and checksums [SS-R03] . . . 11

Figure 7.1 — Building and running one Bluesim bench [SS-R04] . . . 12

Figure 7.2 — make test: the eight Python test files [SS-R05] . . . 12

Figure 7.3 — make test: the nine Bluespec benches [SS-R06] . . . 13

Figure 7.4 — Hardware sweep from segno symbols [SS-R07] . . . 13

Figure 7.5 — Hardware sweep from the model [SS-R08] . . . 13

Figure 7.6 — Final archive listing [SS-R09] . . . 14

Figure 8.1 — Cycle counts from Bluesim [SS-R10] . . . 17

Figure 8.2 — TbCornerCases output [SS-R11] . . . 17

Figure 9.1 — Frequency, DC power and DC design area of the five iterations . . . 19

List of Tables

Table 2.1 — Requirement traceability (functional and verification) . . . 7
Table 3.1 — Segment modes (spec Table 4) . . . 8
Table 4.1 — Sub-systems . . . 9
Table 4.2 — RTL change per synthesis iteration (details and numbers in pd_report.pdf) . . . 10
Table 5.1 — Tools and versions (as used) . . . 10
Table 6.1 — Bluespec sources (bsv/) . . . 11
Table 8.1 — Python test files . . . 14
Table 8.2 — Bluesim benches . . . 14
Table 8.3 — Spec Section 5.3 categories against the tests . . . 15
Table 8.4 — All verification activities . . . 16
Table 9.1 — Post-synthesis (Table 5) and post-place (Table 6) results . . . 18
Table 9.2 — Decision per candidate . . . 18
Table 9.3 — Best design r4h12 at 1.20 ns . . . 18
Table 10.1 — Problems met . . . 19
Table A.1 — All commands of the RTL and verification work . . . 20
Table B.1 — Screenshot checklist . . . 21
Table D.1 — Terms . . . 21

Chapter 1 Introduction

1.1 Background and problem statement

A QR code stores its message as a grid of dark and light modules. A decoder must find the settings in the format information, read and unmask the data modules in a zigzag order, de-interleave them into Reed–Solomon blocks, correct damage, and parse the payload segments. The course asks for this as a synthesizable Bluespec module with a fixed interface, graded by hidden test cases (valid symbols of all versions, damaged symbols, and symbols damaged beyond repair that must raise *failed*).

1.2 Objectives and scope

- Implement `mkQRDecoder` exactly against the supplied interface (QRTypes.bsv, QRTables.bsv unchanged).
- Satisfy functional rules G1–G4 and the interface contract C1–C5.
- Verify thoroughly: software reference model, unit tests, whole-decoder tests, hardware sweeps, one-command regression.
- Take the design through synthesis iterations and place-and-route (reported in pd_report.pdf).

1.3 Report organisation

Chapter 2 analyses the specification and the traceability of every requirement. Chapter 3 gives the theory used. Chapter 4 describes the architecture; Chapter 5 the tools; Chapter 6 the implementation; Chapter 7 every command used; Chapter 8 the verification; Chapter 9 the results; Chapters 10–12 the problems, discussion and conclusion.

Chapter 2 Assignment Specification and Requirements

2.1 What the specification demands

SPECIFICATION SECTION 4.2 — FUNCTIONAL REQUIREMENTS (HIGHLIGHTED)	
G1 Version	Find v from side length N = 17 + 4v; if N is not valid for any $1 \le v \le 40$, **fail**.
G2 Format	Read both 15-bit copies, take the one nearer to a valid value (first copy on a tie); if more than **3** positions away, **fail**.
G3 Correction	Read and unmask data modules, de-interleave, correct each block; if a block cannot be corrected, **fail**.
G4 Payload	Decode segments in order, one byte per character (ASCII for numeric / alphanumeric, the byte itself in byte mode); **fail** on unknown mode, a segment running past the data, or a numeric / alphanumeric group out of range.

SPECIFICATION SECTION 4.3 — INTERFACE AND TIMING CONTRACT (HIGHLIGHTED)	
C1	getByte fires only when a byte is genuinely available; bytes in order, once each; only the last has *last* set.
C2	failed and done are False after reset and while decoding; then exactly one rises and stays until the next start. done only after the last byte is taken (or at once for an empty message). failed may rise at any point.
C3	After failed rises, getByte does not fire until the next start.
C4	mem.req is ready only when a module is wanted; the answer arrives on mem.resp the next cycle and holds; at most one read per cycle.
C5	start and getByte use implicit conditions; no always_ready / always_enabled on either.

SPECIFICATION SECTIONS 6 AND 5 — EVALUATION AND VERIFICATION (HIGHLIGHTED)	
Grading	Functional test cases 10, Verification 5, Design-space exploration 10, Place and route 5 (total 30). Binary diff of the output per test case.
Test cases	Valid codes of all versions; full-size codes with simulated errors; codes with errors beyond the correctable amount (must raise failed).
Verification set (5.3)	Every version, level, mask (not every combination); every mode, multi-segment, empty message, short final groups; exactly floor(e/2) damaged bytes per block, check bytes too, damaged function modules; up to 3 wrong format bits in one or both copies; must-fail cases.
Report must say	what was tested, how many tests ran, what coverage was reached, which bugs the tests caught; **verification/** with a README and one command.
Submission	/ with README.md, report.pdf, pd_report.pdf, bsv/, verification/, pd/ (spec 6.3).

2.2 Test-bench guarantees (A1–A5)

The memory holds the whole symbol before start and does not change (A1); the symbol is upright with no quiet zone (A2); start fires only when idle (A3); damage is flipped modules anywhere including function modules (A4); a block with more than floor(e/2) damaged bytes differs in more than floor(e/2) bytes from every valid block, so a correct decoder detects it and never miscorrects (A5). Consequently the decoder does not locate the symbol and must not reject it because a finder, timing or alignment pattern is damaged.

2.3 Requirement traceability matrix

Table 2.1 — Requirement traceability (functional and verification)

ID	Requirement	Implementation	Verification	Result	Evidence
G1	Version from side length; fail otherwise	QRFrontEnd.bsv size check	test_front_ref (sizes V1–40), bad sizes 20, 22, 178	pass (reference + bench); bench log: see 8.5	Ch. 8
G2	Format nearest-of-32, ≤ 3 errors, first copy on tie	mkQRFrontEnd 2-stage BCH search	exhaustive 2^{15} words (test_front_ref); sweeps	pass	Ch. 8
G3	Unmask, zigzag, de-interleave, RS correct	QRFrontEnd, QRBlocks, QRRS	TbBM, TbRS, TbBlocks; sweeps (damage ≤ t and over t)	pass	Ch. 8
G4	Payload parsing and failure cases	QRPayload.bsv	test_payload_ref (17), TbPayloadModes	pass	Ch. 8
C1–C4	Handshake and timing contract	QRDecoder / QRPayload output stage	test_protocol_spec (19) cycle-level checker	pass	Ch. 8
C5	No always_ready on start / getByte	QRDecoder.bsv interface	port checker in test_protocol_spec	pass	Ch. 8
A4/A5	Damage tolerance and no miscorrection	RS error flags: deg(Λ) > t, root count ≠ deg, root outside block	RS benches (4,594 blocks, partner count), sweeps: 30+30 over-limit cases fail	pass	Ch. 8
V-1	One-command verification, README	verification/Makefile, run_tests.sh, README.md	make test; make full	pass	Ch. 7, 8
PD	Synthesis iterations and PnR	pd/	Table 5 / Table 6	see pd_report.pdf	pd_report
Hidden suite	Functional marks	—	—	**not known until graded**	—

Chapter 3 Theoretical Background

3.1 QR symbol structure

A version-v symbol has N = 17 + 4v modules per side (21×21 to 177×177). Function modules (finder patterns with separators, timing row and column 6, alignment patterns from version 2, version information from version 7, two copies of the format information and the always-dark module at (N−8, 8)) carry no data; all other modules are data modules. A version-1 symbol has 208 data modules, i.e. 26 bytes.

3.2 Format information and BCH(15,5)

The five bits (level 2 bits, mask 3 bits) are protected by a BCH(15,5) code with generator 10100110111 (GF(2)) and XORed with 101010000010010. Only 32 of 2^{15} words are valid and any two differ in at least 7 positions (d_{min} = 7), so the nearest valid word recovers the settings whenever at most 3 of 15 modules are wrong. In hardware the 32 valid words are compared with the read word in parallel by XOR and population count.

3.3 Masks and reading order

One of eight mask patterns is XORed over the data modules; the decoder inverts the module where the mask condition holds (spec Table 2). Data modules are read in two-column strips from the bottom-right corner, moving up then down, right module first, skipping column 6 and every function module. Bits form bytes MSB first.

3.4 Blocks, interleaving and Reed–Solomon

Bytes are divided into blocks; each block is a Reed–Solomon codeword with *e* check bytes and corrects up to *t* = floor(*e*/2) byte errors. Arithmetic is in GF(2^8) with polynomial $x^8+x^4+x^3+x^2+1$ (0x11D), α = 2. The syndromes are

$$\mathbf{S_j = R(\alpha^j)} \text{ for } j = 0 \ldots e-1 \quad \text{(Equation 3.1)}$$

If all are zero the block is clean. Otherwise Berlekamp–Massey finds the error locator Λ(x); a Chien search finds its roots (error positions) and Forney's formula the error values. The block is declared uncorrectable when deg Λ > t, the number of roots differs from deg Λ, or a root lies outside the block. For multi-block symbols the k-th byte read from the symbol is byte i of block b with k = i·n + b (data bytes below d), k = d·n + (b − n_1) (the extra byte of longer blocks), and k = D + j·n + b (check byte j), where n is the number of blocks and D the total data bytes.

3.5 Payload

Table 3.1 — Segment modes (spec Table 4)

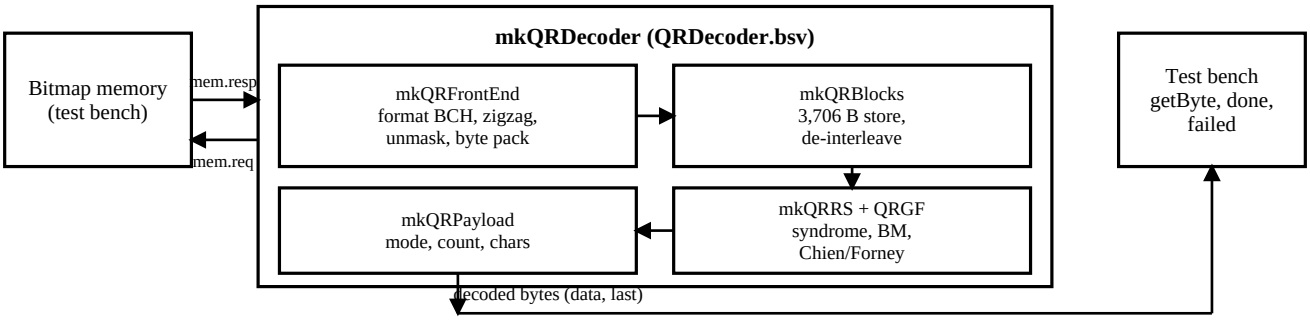
Mode	Indicator	Count width v1–9 / 10–26 / 27–40	Packing
Numeric	0001	10 / 12 / 14	3 digits in 10 bits; 2 digits 7 bits; 1 digit 4 bits
Alphanumeric	0010	9 / 11 / 13	two characters in 11 bits as 45a + b; one character 6 bits
Byte	0100	8 / 16 / 16	one byte in 8 bits
Terminator	0000	—	end of message

Out-of-range groups (three digits above 999, an alphanumeric pair ≥ 2025 or a single value ≥ 45) and truncated segments are malformed and must raise failed.

Chapter 4 System Architecture and Design

4.1 Overall architecture

The decoder is four streaming sub-systems connected by latency-insensitive FIFO / handshake interfaces (Figure 4.1).



start(size) → version v = (size − 17)/4 ; format copies read first, then data modules.

Failure at any stage raises failed; done rises only after the byte with last has been taken.

Figure 4.1 — Block diagram of mkQRDecoder and its surroundings

Table 4.1 — Sub-systems

Module (file)	Function
mkQRFrontEnd (QRFrontEnd.bsv)	Reads the two format copies; nearest-neighbour BCH(15,5) decoding in a 2-stage pipeline (32 parallel XOR + popcount, then registered scan), fail if distance > 3 in both copies; zigzag scan skipping all function patterns (alignment grids from QRTables); unmask; pack MSB-first bytes.
mkQRBlocks (QRBlocks.bsv)	3,706-byte codeword store (version-40 maximum) with 1-cycle registered store pipeline; de-interleaves into up to 81 RS blocks from QRTables partition data; feeds each block to the RS engine; applies corrections in place; streams corrected data bytes onward.
mkQRRS and QRGF (QRRS.bsv, QRGF.bsv)	GF(2^8) arithmetic modulo 0x11D; pipelined 30-syndrome unit; 2-stage Berlekamp–Massey (compile-time selectable 1-stage and 3-stage variants); Chien search and Forney evaluator; uncorrectable flag.
mkQRPayload (QRPayload.bsv)	64-bit shift-register bit buffer with single-cycle variable-width extraction; mode and count parsing with version-dependent widths; range checks; emits characters with exact last-byte signalling.
mkQRDecoder (QRDecoder.bsv)	Top module, exactly the supplied interface; control of start / done / failed.

4.2 Data and control flow

start(size) latches the size and derives the version. The front end first reads the format bits (both copies) and then streams the data modules through mem.req / mem.resp, one request per cycle, unmasking and packing bytes. The store is written as bytes arrive; after the last byte the block engine corrects blocks one after another and releases data bytes to the payload parser. The payload parser outputs through getByte under back-pressure: it holds its output until the test bench takes it. A fail from any stage raises failed and suppresses further output (C3).

4.3 Design decisions

- **Read only the format and data modules**: the symbol is never located or validated, so damaged finders or timing patterns cannot cause rejection (A2, A4).
- **Reciprocal multiply-shift instead of dividers** (x171 >> 9, x91 >> 12, x41 >> 12) in the front end and payload to shorten the critical path.
- **One shared RS engine**, blocks processed sequentially: small area, latency proportional to the number of blocks.
- **Compile-time variants** (`-D QR_R0_PLAIN`, `QR_R2_BM`, `QR_R4_BM + QR_STORE_PIPE`) reproduce the earlier synthesis iterations; the default build is the fast variant used in the final design.
- **Store not cleared between symbols**: a new symbol overwrites it; this is not observable at the interface.

4.4 Micro-architectural evolution across iterations

Table 4.2 — RTL change per synthesis iteration (details and numbers in pd_report.pdf)

Iteration	RTL change	Clock (ns)
1 r0_plain	single-cycle BM, un-gated store (baseline)	5.0
2 r2	BM in 3 phases, clock gating	5.0
3 r4_2p5	2-stage BM, 1-cycle store pipeline	2.5
4 r4g20	divider-free arithmetic, pipelined BCH search and zigzag	2.0
5 r4h12 (best)	further backend timing fixes	1.2

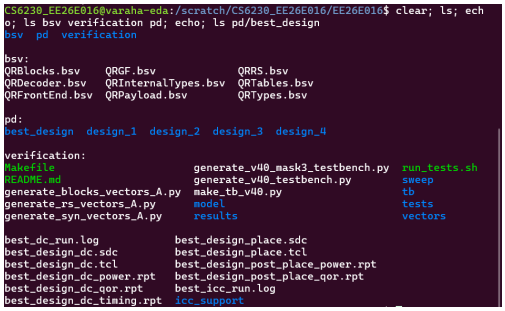


Figure 4.2 — Project directory structure [SS-R01]

Chapter 5 Development Environment and Tools

Table 5.1 — Tools and versions (as used)

Item	Value
Server	varaha-eda, IP 10.21.227.38; project path /scratch/CS6230_EE26E016/
Bluespec compiler / simulator	bsc 2021.12.1 (build fd501401), Bluesim (bsc -sim); partner's local bsc 2026.07.1 gives 38,069 cycles for V40 (5-cycle reset offset)
Python	python3 with the reference models and the segno 1.6.6 QR encoder (cross-check with zxing-cpp on 90 of 90 symbols)
Build	GNU make, bash, zip, md5sum
Synthesis / PnR	Design Compiler U-2022.12, IC Compiler V-2023.12, SAED14nm RVT (see pd_report.pdf)

The submission follows the layout of spec Section 6.3. The verification directory contains the Python models, tests, Bluespec benches, vectors, sweep generators and results.

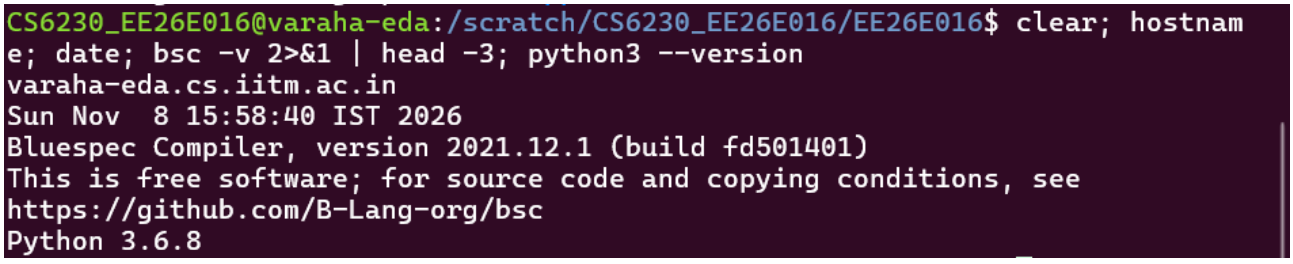


Figure 5.1 — Tool versions on the server [SS-R02]

Chapter 6 Implementation

6.1 Source-file inventory

Table 6.1 — Bluespec sources (bsv/)

File	Lines (working tree)	Role	MD5 (submitted)
QRDecoder.bsv	101	top module, interface	f7552234da53808270ef08e6dcc804ee
QRFrontEnd.bsv	545	format decode, zigzag, unmask	e68f4476ca56676b1be854a1295aef71
QRBlocks.bsv	861	store, de-interleave, RS control	24abd4c7bb98e19b5e98f6d3e1e21b83
QRRS.bsv	806	syndrome, BM, Chien / Forney	ff3cd387c4afcd3ddc633abdaa214430
QRGF.bsv	51	GF(2^8) arithmetic	46e86b49023f3fd5be6642b07e510f22
QRPayload.bsv	453	payload parser	1f8218653528764a9b7fc9abd4ca8801
QRInternalTypes.bsv	57	internal types	645e5ec01863410a6cdb01a3ae6a025c
QRTypes.bsv	50	supplied (unmodified)	c28f426163a3ea8ebf947ebc432e251e
QRTables.bsv	576	supplied (unmodified)	a8f0c0d248e87f33231cd23622aa6dde

The nine files in the submission have the MD5 sums listed; the same sums were confirmed for the partner's copy, so both of us hold identical RTL. The build uses only these files.

6.2 Module interfaces

```
interface QRDecoder;
   method Action               start(Coord size);
   method ActionValue#(OutByte) getByte();
   method Bool                 failed();
   method Bool                 done();
   interface MemPort           mem;     // req: (row, col); resp: Bit#(1)
endinterface
```

start and getByte carry implicit conditions (C5); mem.resp is always enabled and always ready as in the supplied QRTypes.bsv.

6.3 Key implementation points

- **Format decode**: both copies are read; each is compared with all 32 valid codewords (XOR + popcount); the better copy wins, the first on a tie; distance > 3 on both copies fails.
- **Zigzag and function-module skipping**: coordinates advance in the strip order of Section 1.3; a function-module predicate uses the alignment-centre table of the version and the version-information blocks for v ≥ 7.
- **Store pipeline**: reads and writes of the 3,706-byte store are registered, trading one cycle of latency for timing.
- **Berlekamp–Massey in two stages** and Chien / Forney pipeline: uncorrectable detection by degree, root count and root range.
- **Payload buffer**: a 64-bit shift register allows variable-width extraction in one cycle; numeric and alphanumeric ranges are checked, giving failed on out-of-range groups.

The source files are the authoritative detail; this report intentionally does not repeat them line by line. The complete listing is part of the submitted `bsv/` directory.

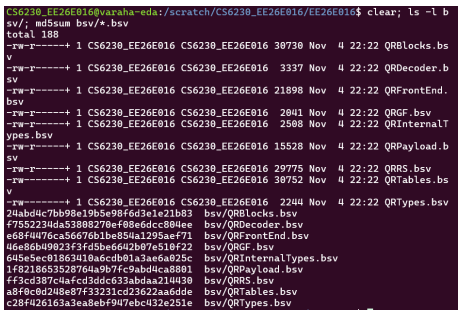


Figure 6.1 — Sources and checksums [SS-R03]

Chapter 7 Command-by-Command Workflow

Every command below is listed with its working directory and purpose. **verified**: text of the script or a run recorded in this project; **reconstructed**: rebuilt from scripts and run names, to be confirmed against the shell history. Output evidence is given as screenshot slots.

7.1 Build commands

```
# CMD-001  Verilog generation (as used inside run_dc_full.sh)          [verified]
cd build_<TAG>
bsc -u -verilog -vdir . -bdir . -info-dir . -remove-dollar -g mkQRDecoder QRDecoder.bsv

# CMD-002  Clean build of the submission RTL against supplied files       [verified]
mkdir -p build_v build_b
bsc -u -verilog -vdir build_v -bdir build_b -p +:bsv -g mkQRDecoder bsv/QRDecoder.bsv
```

CMD-001 flags: `-u` recompile out-of-date packages; `-verilog` generate Verilog; `-vdir/-bdir/-info-dir` output folders; `-remove-dollar` remove $ from names for DC; `-g mkQRDecoder` generate that module. **CMD-002** adds `-p +:bsv` so the supplied QRTypes / QRTables are found in bsv/.

7.2 Bluesim benches

```
# CMD-003  build and run one bench (loop over the nine benches in run_tests.sh)   [verified]
cp bsv/QR*.bsv verification/tb/TbVersion40Mask3.bsv  $W/
cd $W
bsc -u -sim -bdir o -simdir o -info-dir o -g mkTbVersion40Mask3 TbVersion40Mask3.bsv
bsc -sim -e mkTbVersion40Mask3 -bdir o -simdir o -o run
(cd verification/vectors/backend && $W/run)     # benches read their vector files from the ▫
    current directory
```

The nine benches are TbGF, TbGFVectors, TbRS, TbSyndrome, TbBM, TbBlocks, TbCornerCases, TbPayloadModes and TbVersion40Mask3. A bench passes when its log is non-empty and contains none of `RESULT: FAIL, MISMATCH, [FAIL], ERROR, Error:, error:, TESTS FAILED, assertion`. Only `QR*.bsv` is copied from bsv/, so no stale bench can shadow the one in tb/.

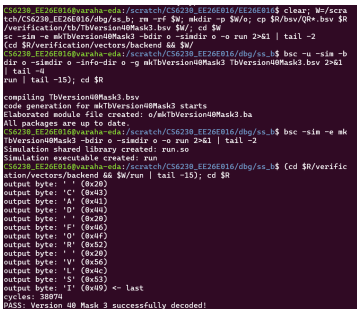


Figure 7.1 — Building and running one Bluesim bench [SS-R04]

7.3 One-command regression

```
# CMD-004  quick regression: 8 Python test files + 9 Bluespec benches        [verified]
cd EE26E016/verification && make test
# CMD-005  full regression: quick + two 120-case hardware sweeps in 4 shards [verified]
cd EE26E016/verification && make full
```

The Python tests run as `python3 tests/<file>` from the verification folder (with a 600 s timeout and stdin from /dev/null). The _build folder is removed only when everything passed.

```
CS6230_EE26E016@varaha-eda:/scratch/CS6230_EE26E016/EE26E016/verification$ make test 2>&1 | tail -30

bash run_tests.sh quick
== 1. Python reference models and their self-tests
test_against_segno.py                       PASS
test_backend.py                             PASS
test_decode_ref.py                          PASS
test_front_ref.py                           PASS
test_gf_ref.py                              PASS
test_model_interface.py                     PASS
test_payload_ref.py                         PASS
test_protocol_spec.py                       PASS
```

Figure 7.2 — make test: the eight Python test files [SS-R05]

```
CS6230_EE26E016@varaha-eda:/scratch/CS6230_EE26E016/EE26E016/verification$ make test 2>&1 | tail -30

== 2. Bluespec test benches (GF, RS, syndrome, Berlekamp-Massey, de-interleave, corner cases, payload modes, versi
on 40)
TbGF                                    PASS (2 lines)
TbGFVectors                             PASS (2 lines)
TbRS                                    PASS (5 lines)
TbSyndrome                              PASS (3 lines)
TbBM                                    PASS (4 lines)
TbBlocks                                PASS (5 lines)
TbCornerCases                           PASS (41 lines)
TbPayloadModes                          PASS (25 lines)
TbVersion40Mask3                        PASS (21 lines)

ALL VERIFICATION STEPS PASSED
CS6230_EE26E016@varaha-eda:/scratch/CS6230_EE26E016/EE26E016/verification$
CS6230_EE26E016@varaha-eda:/scratch/CS6230_EE26E016/EE26E016/verification$
```

Figure 7.3 — make test: the nine Bluespec benches [SS-R06]

7.4 Hardware sweeps

```
# CMD-006  generate 120 cases from the model                                  [verified]
python3 verification/sweep/gen_sweep.py verification/sweep/cases_model 120
# CMD-007  generate 120 independent cases from segno symbols                  [verified]
python3 verification/sweep/gen_sweep_segno.py verification/sweep/cases 120
# CMD-008  run in 4 shards (full decoder in Bluesim), then check               [verified]
bash verification/sweep/run_sweep.sh <cases_dir> <shard> 4          # shard = 0..3
python3 verification/sweep/check_sweep.py <cases_dir>
```

```
CS6230_EE26E016@varaha-eda:/scratch/CS6230_EE26E016/EE26E016$ clear; cd $R/v
erification/sweep; python3 check_sweep.py cases 2>&1 | tail -12
 2>&1 | tail -12
; cd $Rcases: 120
  limit PASS     60
  none PASS      30
  over PASS      30
  (over: hardware reported failed: 30)
MISMATCHES: 0
```

Figure 7.4 — Hardware sweep from segno symbols [SS-R07]

```
CS6230_EE26E016@varaha-eda:/scratch/CS6230_EE26E016/EE26E016/verification/sw
eep$ python3 check_sweep.py cases_model 2>&1 | tail -12; cd $R
cases: 120
  limit PASS     60
  none PASS      30
  over PASS      30
  (over: hardware reported failed: 30)
MISMATCHES: 0
CS6230_EE26E016@varaha-eda:/scratch/CS6230_EE26E016/EE26E016$
```

Figure 7.5 — Hardware sweep from the model [SS-R08]

7.5 Packaging and integrity

```
# CMD-009  final packaging (script of the project)                            [verified]
bash pack2.sh
# CMD-010  integrity and archive                                              [verified]
md5sum bsv/*.bsv
zip -r EE26E016.zip EE26E016
# CMD-011  copy to the PC (run on the PC)                                     [verified]
scp <user>@10.21.227.38:/scratch/CS6230_EE26E016/EE26E016.zip .
```

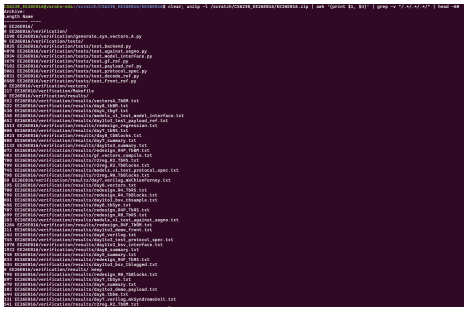


Figure 7.6 — Final archive listing [SS-R09]

Physical-design commands (dc_shell, icc_shell and their drivers) are listed in Appendix A of pd_report.pdf.

Chapter 8 Verification and Testing

8.1 Methodology

Verification follows the spec route (Section 5): a software reference model first, then unit tests against the matching model function, then whole-decoder tests in Bluesim driven by vectors written by the model, then randomised hardware sweeps. The model author of a unit is not the author of its RTL. Every bug found was turned into a test.

8.2 Tier 1: Python reference and protocol tests (8 files, 76 tests)

Table 8.1 — Python test files

File	Tests	Scope
test_against_segno.py	3	model vs the independent segno encoder: all 40 versions × 4 levels, masks 0–7, round trips
test_decode_ref.py	7	top model: handout V3-M sample, format errors ≤ 3, RS ≤ t, uncorrectable format / RS, bad size, finder damage
test_front_ref.py	16	sizes V1–40, **exhaustive 2^{15} BCH words**, copy-1 / copy-2 rules, masks, alignment grids, zigzag
test_model_interface.py	6	codeword totals vs QRTables, front-end pass / fail hand-off, payload tuple contract
test_payload_ref.py	17	numeric / alphanumeric / byte, count widths for the three version ranges, multi-segment, empty, range and truncation errors
test_protocol_spec.py	19	cycle-level checker of C1–C5; catches 14 illegal done / failed / last transitions; port checker
test_backend.py	5	partition tables, de-interleaver, RS correct ≤ t and detect > t
test_gf_ref.py	3	exhaustive 65,536 GF multiplications
Total	**76**	

8.3 Tier 2: Bluespec benches (9 in make test)

Table 8.2 — Bluesim benches

Bench	Unit under test	Stimulus	Status
TbGF, TbGFVectors	QRGF	all 65,536 GF(2^8) input pairs against golden hex	pass, 0 mismatches
TbSyndrome, TbBM, TbRS	syndrome, BM, RS	randomised and exhaustive-error-count blocks (0…t errors and > t)	pass, 0 mismatches
TbBlocks	mkQRBlocks	multi-block symbols V1–40 incl. over-limit and mid-stream restarts	pass, 0 mismatches
TbPayloadModes	mkQRDecoder	V1-L numeric, V1-Q alphanumeric, V2-M mixed, V5-Q four blocks, V1-H t = 8	pass (5 cases)
TbVersion40Mask3	mkQRDecoder	177×177 V40-L, mask 3, 2,953 payload bytes	pass, 38,074 cycles
TbCornerCases	mkQRDecoder	bad sizes (20, 22, 178), copy-2 fallback, dual-copy fail, empty message, bad mode, numeric ≥ 1000, finder damage	**passes in make test; the archived partner log is being re-generated (see 8.5)**

Further benches (TbLogged, TbDamaged3, TbDamaged6, TbSample, TbVersion40) are used for the cycle counts and the handout sample. Block and symbol counts quoted for TbRS / TbBlocks (4,594 blocks; 727 symbols) are the partner's counts and are

reproduced by running the benches.

8.4 Tier 3: whole-decoder hardware sweeps

RESULT — HARDWARE SWEEPS (FULL DECODER IN BLUESIM)	
Sweep 1: segno symbols	**120 / 120, 0 mismatches** (30 clean, 60 damaged to within the limit decode exactly, 30 over the limit report failed); cross-checked with zxing-cpp on 90 / 90
Sweep 2: model symbols	**120 / 120, 0 mismatches** (same 30 / 60 / 30 mix)
Total	**240 symbols, 0 mismatches**; all EC levels, mixed modes

8.5 Coverage of the required test categories

Table 8.3 — Spec Section 5.3 categories against the tests

Required category	Covered by
Every version, level, mask (not every combination)	test_against_segno (all 40 × 4; masks 0–7), sweeps
Every mode; multi-segment; empty; short final groups	test_payload_ref, TbPayloadModes, TbCornerCases
Exactly floor(e/2) damaged bytes per block; check bytes; function modules	TbRS, TbBlocks, sweeps (damage to limit), finder damage tests
Up to 3 wrong format bits in one or both copies	exhaustive BCH test, test_decode_ref, TbDamaged3
Must-fail: one byte too many, format past repair, bad size, malformed payload	sweeps (30 + 30 over-limit), TbCornerCases, TbDamaged6, test_payload_ref
Stalled getByte, several symbols without reset, C1–C5	protocol checker, TbBlocks restarts; bench back-pressure

8.6 Complete verification inventory

The table lists every verification activity carried out for the submission, with its outcome. All items were run on the final submitted tree; the logs are in `verification/results/`.

Table 8.4 — All verification activities

#	Activity	What it covers	Outcome
1	Python reference model vs segno	40 versions × 4 levels, masks 0–7, round trips	3 / 3 pass
2	Top reference model tests	handout sample, format errors ≤ 3, RS ≤ t, uncorrectable cases, bad size, finder damage	7 / 7 pass
3	Front-end model tests	sizes V1–40, exhaustive 2^{15} format words, copy rules, masks, alignment grids, zigzag	16 / 16 pass
4	Model interface tests	codeword totals vs QRTables, hand-off contracts	6 / 6 pass
5	Payload model tests	all modes, count widths, multi-segment, empty, range and truncation errors	17 / 17 pass
6	Protocol checker (C1–C5)	cycle-level handshake rules, port attributes, 14 illegal transitions detected	19 / 19 pass
7	Back-end model tests	partition tables, de-interleaver, RS correct / detect	5 / 5 pass
8	GF(2^8) model tests	exhaustive 65,536 multiplications	3 / 3 pass
9	Bluesim TbGF, TbGFVectors	GF arithmetic RTL, all 65,536 pairs vs golden	pass, 0 mismatches
10	Bluesim TbSyndrome, TbBM, TbRS	syndrome, Berlekamp–Massey, RS correction with 0..t and > t errors	pass, 0 mismatches
11	Bluesim TbBlocks	de-interleave and RS over V1–40, over-limit symbols, mid-stream restarts	pass, 0 mismatches
12	Bluesim TbPayloadModes	numeric, alphanumeric, mixed, 4-block, t = 8	5 / 5 pass
13	Bluesim TbCornerCases	bad sizes, copy-2 fallback, dual-copy fail, empty message, bad mode, numeric ≥ 1000, finder damage	passes in make test and in the partner's script run against the submitted bsv/ (10/10, log saved in verification/results/rishi_benches/)
14	Bluesim TbVersion40Mask3 / TbVersion40	177×177 symbol, 2,953 bytes	pass; 38,074 cycles
15	Bluesim TbLogged / TbDamaged3 / TbDamaged6	handout V3-M clean, 3 RS errors + format damage, 6 errors > t rejected	pass; 563 / 642 cycles; rejected
16	Hardware sweep, segno symbols	120 full-decoder cases: 30 clean, 60 within limit, 30 over limit	120 / 120, 0 mismatches
17	zxing-cpp cross-check of segno cases	independent decoder on the sweep symbols	90 / 90 agree
18	Hardware sweep, model symbols	120 cases, same mix	120 / 120, 0 mismatches
19	R5 backend functional check (experimental)	4-bank store variant on the hardware sweeps	240 / 240 pass
20	Source identity check	md5 of the nine bsv files equal in both partners' copies	identical
21	Security review of the RTL	only course ports, no system tasks, only library cells, bounds-checked payload	no issue found; store not cleared between symbols (not observable)
22	Number cross-check	all cycle counts and PD numbers compared with server reports	two typos in the partner draft found and corrected
23	One-command regression	make test (quick) and make full (adds the sweeps)	make test on the final tree: ALL VERIFICATION STEPS PASSED (8 Python files, 9 benches); **make full on the final tree: ALL VERIFICATION STEPS PASSED (10 min 7 s; log verification/results/make_full_final.log)**
24	Run of the partner's three benches against the submitted bsv/	TbCornerCases, TbPayloadModes, TbVersion40Mask3 logs	**passed**: TbCornerCases 10/10, TbPayloadModes 5/5, TbVersion40Mask3 decoded in 38,074 cycles (logs in verification/results/rishi_benches/)
25	Clean rebuild from the fresh handout with our bsv/ copied over	spec Section 6.3 check that the uploaded files are the tested files	**passed**: handout copy with our bsv/ built and ran TbSample, decoded HELLO CAD FOR VLSI in 563 cycles; QRTypes.bsv and QRTables.bsv are identical to the handout's

8.7 Known limits of the verification

- The sweeps do not damage format-information bits; format damage is covered by the exhaustive BCH test and directed benches.
- segno does not produce multi-segment symbols; multi-segment is covered by model-made vectors and TbPayloadModes.
- The archived TbCornerCases log of the partner does not match the bench file; only results reproduced by running the benches are claimed here.
- Coverage tooling (Verilator --coverage) is optional in the spec and was not used; category coverage is shown in the coverage table above.
- The hidden grading suite is not available; passing it is not claimed.

```
CS6230_EE26E016@varaha-eda:/scratch/CS6230_EE26E016/EE26E016$ grep -rn -E "563|642|38074" verification/results 2>/
dev/null | head -10
verification/results/full_decoder/sim_TbDamaged3.log:276:@563 byte 49 1
verification/results/full_decoder/sim_TbDamaged3.log:278:decoded in 563 cycles
verification/results/full_decoder/summary.txt:5:TbDamaged3  want ok   -> output 'HELLO CAD FOR VLSI'  protocol OK
 decode @0: latency 563 cycles (start to last byte taken), 238 memory reads
verification/results/redesign_regression.txt:10:TbLogged     want ok   -> output 'HELLO CAD FOR VLSI'  protocol OK
 decode @0: latency 563 cycles (start to last byte taken), 238 memory reads
verification/results/redesign_regression.txt:11:TbDamaged3  want ok   -> output 'HELLO CAD FOR VLSI'  protocol OK
 decode @0: latency 642 cycles (start to last byte taken), 238 memory reads
verification/results/redesign_regression.txt:13:TbVersion40 want ok   -> SUCCESS: Version 40 decoded in 38074 cycl
es
verification/results/redesign_regression.txt:14:TbV40Mask3  want ok   -> PASS: Version 40 Mask 3 successfully deco
ded!  cycles: 38074  text starts 'HELLO CAD FOR VLSI'
verification/results/full_decoder_r4P/sim_TbDamaged3.log:254:@563 byte 43 0
verification/results/full_decoder_r4F/sim_TbDamaged3.log:276:@642 byte 49 1
verification/results/full_decoder_r4F/sim_TbDamaged3.log:278:decoded in 642 cycles
CS6230_EE26E016@varaha-eda:/scratch/CS6230_EE26E016/EE26E016$
```

Figure 8.1 — Cycle counts from Bluesim [SS-R10]

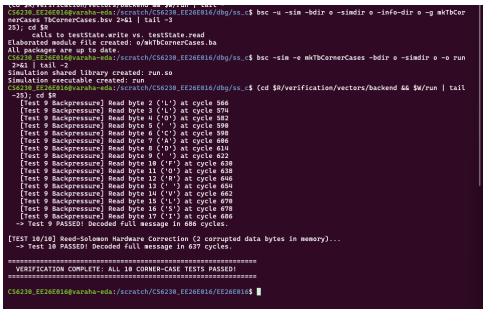


Figure 8.2 — TbCornerCases output [SS-R11]

Chapter 9 Results and Analysis

RESULT — FUNCTIONAL AND VERIFICATION (HIGHLIGHTED)	
Python tests	**76 / 76 pass** (8 files)
Bluespec benches	**9 / 9 pass** under make test
Hardware sweeps	**240 / 240 correct, 0 mismatches** (2 × 120)
Latency V3-M (handout)	**563 cycles** = 0.676 µs at 1.20 ns; with 3 RS errors and damaged format: 642 cycles
Latency V40-L	**38,074 cycles** = 45.69 µs at 1.20 ns (12.89 cycles per payload byte)

RESULT — PHYSICAL DESIGN (HIGHLIGHTED; FULL DETAIL IN PD_REPORT.PDF)	
Table 5	**5.00 → 1.20 ns, 200 → 833.3 MHz (4.17×)**; f2f 4.98 → 1.19 ns; 0 violating paths in every run
Table 6	**best design r4h12 @ 1.20 ns**: 0 setup / 0 hold violations, **92.29%** utilisation, 243.98 × 243.45 µm, 52,285.97 µm², 7.3985 mW

9.1 All timing results (ns) — summary table

Table 9.1 — Post-synthesis (Table 5) and post-place (Table 6) results

Design	Tag	Clock T (ns)	f2f path (ns)	Slack (ns)	Fmax (MHz)	Leaf cells	Seq. cells	Power (mW)	Area (μm²)
1	r0_plain	5.00	4.98	0.00	200.0	115,224	32,102	9.9178	216,341.52
2	r2	5.00	4.98	0.00	200.0	90,815	36,247	4.5249	141,711.74
3	r4_2p5	2.50	2.48	0.00	400.0	92,463	35,744	8.8798	140,112.70
4	r4g20	2.00	1.97	0.00	500.0	89,407	36,657	11.9537	151,986.89
5 (best)	r4h12	1.20	1.19	0.00	833.3	92,233	36,667	19.8907	143,448.02
best, placed	r4h12 ICC	1.20	0.80	0.00	833.3	85,490	36,667	7.3985	52,285.97
r4h11 (aux.)	DC	1.10	1.08	0.00	909.1	93,536	36,667	21.6118	144,886.2
R5 (exp.)	DC	0.70	0.69	0.00	1428.6	92,267	37,020	10.1773	137,036.54
R5 (exp.)	DC	0.60	—	0.00	1666.7	—	—	11.8705	140,700.47
R5 (exp.)	DC	0.55	0.54	0.00	1818.2	97,050	37,020	13.0061	140,392.13

Rows for 1.10 ns and R5 (0.70–0.55 ns) are auxiliary DC results, not Table 5 entries. DC and ICC power / area are not comparable. The 3.0 ns run of design 1 and the R5 placements are in Table 4.2 of pd_report.pdf.

9.2 Why the faster periods (down to 0.55 ns) were not chosen

Table 9.2 — Decision per candidate

Candidate	Period (ns)	Why not chosen / why chosen
Designs 1–4	5.0, 5.0, 2.5, 2.0	met timing but are slower; each was superseded by the next architectural step
r4h11	1.10	viable alternative: places clean at 1.10 ns (cores 242, 244), 9.1% faster but about 9% more dynamic power and 0.65% more placed area than r4h12; r4h12 kept for lower power, larger margin (0.40 vs 0.35 ns f2f) and a third clean core size
R5 4-bank store	0.70 / 0.60 / 0.55	meets DC, but none of 12 placements (0.55–0.70 ns) was clean (1–14 f2f violations); larger placed area, higher power; its RTL and model share one author, against spec Section 5.2
r4h12 (chosen)	**1.20**	**fastest period that places clean (0 setup / 0 hold) at four core sizes; independent RTL / model; verified functionally**

9.3 Best-design data sheet

Table 9.3 — Best design r4h12 at 1.20 ns

Item	Value
Clock period / frequency	1.20 ns / 833.3 MHz, 40% external delay (0.48 ns)
f2f critical path	1.19 ns after DC (16 logic levels); 0.80 ns after placement (7 levels)
Slack / violations	0.00 ns; 0 setup, 0 hold
Leaf / sequential cells	92,233 / 36,667 (DC); 85,490 / 36,667 (placed)
Cell area	55,777.5 μm² (DC); 52,285.97 μm² (placed)
DC design area	143,448.02 μm²
Power	19.8907 mW (DC); 7.3985 mW (placed: 6.3528 internal + 0.9290 switching + 0.1167 leakage)
Block / utilisation	243.98 × 243.45 μm; 92.29%
Cycles	V3-M 563; V40-L 38,074
Wall time	V3-M 0.676 μs; V40-L 45.69 μs (cycles × 1.20 ns)

Wall time is a calculated value: time = cycles × clock period (Equation 9.1).

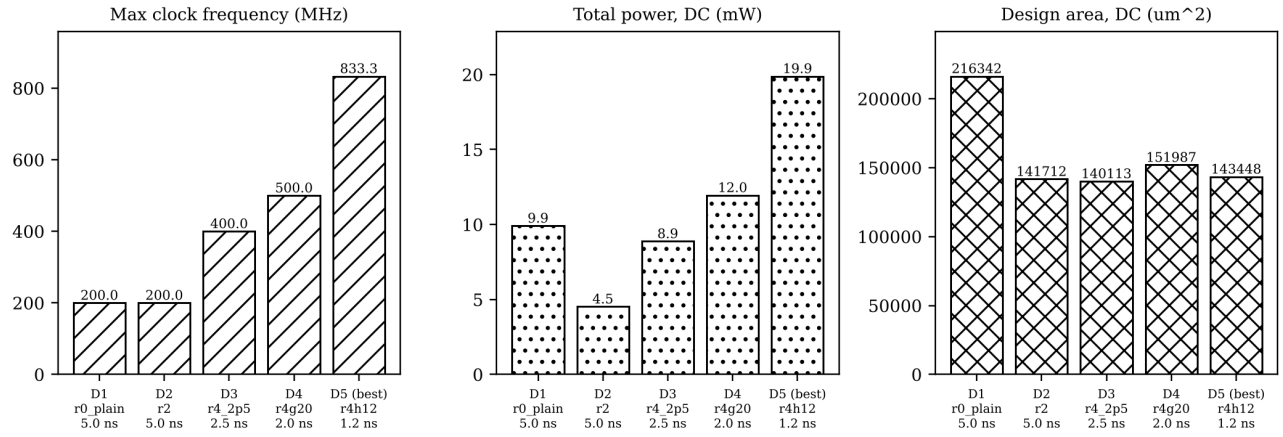


Figure 9.1 — Frequency, DC power and DC design area of the five iterations

9.4 Discussion of the results

The architecture changes bought a 4.17× faster clock with a smaller standard-cell area (-7.3%). Clock gating halves the power at equal frequency, while the highest clock costs the most power in DC. The functional evidence is consistent across three independent sources (reference tests, benches, two sweeps), and all over-limit cases fail rather than miscorrect, as A5 requires.

Chapter 10 Challenges, Debugging and Solutions

Table 10.1 — Problems met

Problem / error	Root cause	Fix
gen_sweep.py: “Unknown version 1 or level 1”	block table is read only from ../../bsv/QRTables.bsv	run from the submission tree (or copy the file locally)
Benches fail to find vector files	benches read hex files from the current directory	run them from verification/vectors/backend
Wrong bench executed	old Tb files in the working bsv/ shadowed tb/	copy only QR*.bsv into the build folder
False FAIL on titles containing “Format Bit Error”	pass rule matched the word Error	rule changed to the specific failure strings
model_before_rishi folder hijacked model discovery	stale folder inside verification/	moved to _archive
Permission denied on /scratch/dbg	wrong scratch location	use /scratch/CS6230_EE26E016/dbg
Stale run_tests.sh / README uploaded repeatedly	old copies on the server	checked file sizes before each run (final 2,543 and 3,397 bytes)
Wrong numbers in the partner draft (power 8.8785, 11.9544; “13” placements)	copy / count errors	corrected from server reports (8.8798, 11.9537; 12 runs)
make test: 3 Python tests FAILED in the packaged tree (test_decode_ref, test_front_ref, test_model_interface)	tests read the handout TbSample.bsv from bsv/, which holds only our nine files in the submission	tests now read verification/tb/TbSample.bsv; make test then passed 8 / 8 Python files and 9 / 9 benches (found on the final-tree check, 8 Nov)
V40 cycle count differs with a newer bsc	5-cycle reset offset in bsc 2026.07.1	report the cycle counts of the course version 2021.12.1 (38,074), note 38,069

RTL bugs found by the verification are not enumerated here because no separate bug log was kept; every defect found during development was added to the test suite as a regression test (spec Section 5.4). *[Authors: add dated entries from the project notes if available.]*

Chapter 11 Discussion

Achieved. All functional rules G1–G4 and contract rules C1–C5 are implemented and checked by the reference tests, benches and sweeps. The design reaches 833.3 MHz in synthesis and places cleanly.

Strengths. Independent verification (model written by a different person than the RTL, second sweep from an external encoder), one-command regression, exhaustive tests of the small units (GF multiplier, format decoder), cycle-level protocol checker.

Limitations. The format-information damage is not part of the sweeps; the hidden test suite is unseen; the layout is only placed (PlaceOpt), not routed; latency for a V40 symbol is dominated by the one-module-per-cycle read (about 31,000 module reads) followed by sequential block correction.

Trade-offs. A single shared RS engine saves area at the cost of latency proportional to the number of blocks; the 3,706-byte register store dominates the sequential-cell count (about 36,000 flops).

Chapter 12 Conclusion and Future Work

FINAL SUMMARY

mkQRDecoder implements the full specification (versions 1–40, all levels, masks and modes). **76 Python tests, 9 Bluespec benches and 240 hardware-sweep symbols pass with 0 mismatches**; V40 decodes in **38,074 cycles (45.69 µs at 1.20 ns)**. Synthesis improved from 5.00 ns to **1.20 ns (833.3 MHz)**; the best design places with **0 violations, 92.29% utilisation, 52,285.97 µm², 7.3985 mW**.

Future work: sweeps that also damage the format bits and produce multi-segment symbols; Verilator coverage; a parallel multi-engine RS for lower latency; clock-tree synthesis and routing with DRC/LVS sign-off.

Appendix A Complete Command Reference

Table A.1 — All commands of the RTL and verification work

ID	Command	Directory	Purpose	Status
CMD-001	bsc -u -verilog ... -g mkQRDecoder QRDecoder.bsv	build_<TAG>	Verilog for synthesis	verified
CMD-002	bsc -u -verilog -vdir build_v -bdir build_b -p +:bsv -g mkQRDecoder bsv/QRDecoder.bsv	EE26E016	clean RTL compile	verified
CMD-003	bsc -u -sim ...; bsc -sim -e mkTb... -o run; ./run	work dir, run from vectors/backend	build and run a bench	verified
CMD-004	make test	verification/	quick regression	verified
CMD-005	make full	verification/	quick + sweeps	verified
CMD-006	python3 verification/sweep/gen_sweep.py <dir> 120	EE26E016	model sweep cases	verified
CMD-007	python3 verification/sweep/gen_sweep_segno.py <dir> 120	EE26E016	segno sweep cases	verified
CMD-008	bash verification/sweep/run_sweep.sh <dir> <shard> 4; python3 .../check_sweep.py <dir>	EE26E016	run and check sweeps	verified
CMD-009	bash pack2.sh	server	assemble submission	verified
CMD-010	md5sum bsv/*.bsv; zip -r EE26E016.zip EE26E016	server	integrity, archive	verified
CMD-011	scp ... EE26E016.zip .	PC	download	verified
CMD-012	dc_shell / icc_shell drivers	pd/	see pd_report.pdf Appendix A	see there

Appendix B Screenshot Evidence Index

Save all screenshots in `/mnt/c/Users/KOTAPATI KISHORE/Documents/CS6230_screenshots/` with the names below; rebuild the report and the slots are filled automatically.

Table B.1 — Screenshot checklist

ID	File name	Content	Section	Need	Status
SS-R01	SS-R01_project_tree.png	Project directory structure	4.1	required	inserted
SS-R02	SS-R02_tool_versions.png	Tool versions on the server	5.1	optional	inserted
SS-R03	SS-R03_bsv_sources.png	Sources and checksums	6.1	required	inserted
SS-R04	SS-R04_bench_build_run.png	Building and running one Bluesim bench	7.2	required	inserted
SS-R05	SS-R05_make_test_python.png	make test: the eight Python test files	7.3	required	inserted
SS-R06	SS-R06_make_test_benches.png	make test: the nine Bluespec benches	7.3	required	inserted
SS-R07	SS-R07_sweep_segno.png	Hardware sweep from segno symbols	7.4	required	inserted
SS-R08	SS-R08_sweep_model.png	Hardware sweep from the model	7.4	required	inserted
SS-R09	SS-R09_final_zip_listing.png	Final archive listing	7.5	required	inserted
SS-R10	SS-R10_bsim_cycles.png	Cycle counts from Bluesim	8.5	required	inserted
SS-R11	SS-R11_corner_cases.png	TbCornerCases output	8.5	required	inserted

Appendix C File and Directory Inventory

```
EE26E016/
  README.md  report.pdf  pd_report.pdf
  bsv/            QRDecoder QRFrontEnd QRBlocks QRRS QRGF QRPayload QRInternalTypes QRTypes ▫
      QRTables (.bsv)
  verification/   model/ tests/ tb/ vectors/ sweep/(cases, cases_model, tools) results/ ▫
      Makefile run_tests.sh README.md
  pd/             design_1 .. design_4  best_design (with icc_support/)
```

Appendix D Glossary and Abbreviations

Table D.1 — Terms

Term	Meaning
BCH	Bose–Chaudhuri–Hocquenghem code protecting the format bits
BM	Berlekamp–Massey algorithm
Chien search	root search of the error locator
Codeword	one byte of the symbol's data stream
EC level	error-correction level L, M, Q, H
Forney	formula for the error values
GF(2^8)	Galois field with 256 elements, polynomial 0x11D
Syndrome	$S_j = R(\alpha^j)$, zero for a clean block
t	floor(e/2) correctable bytes per block
Bluesim	Bluespec's cycle-accurate simulator

Appendix E References

- [1] CS6230 CAD for VLSI, *Project: QR Code Decoder* (specification), 15 Oct 2026.
- [2] ISO/IEC 18004, *QR Code bar code symbology specification*.
- [3] Bluespec, Inc., *BSV reference guide* and bsc 2021.12.1 documentation.
- [4] segno 1.6.6 QR encoder (used for independent test symbols).
- [5] pd_report.pdf of this submission (physical design).